

08_reversals

```
source("R/00_setup.R")
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
c2 <- readRDS("c2_matched.rds")
```

```
#' The verified reversal set. Stage 2 of the classification yielded seven  
#' candidates; the set below is the post-verification classification after  
#' inspecting trajectory geometry against the independent Welzel indices.  
#' Edit here if the verified set changes, and record the change on OSF.
```

```
REV_ISO <- c("IDN", "IND", "JOR", "JPN", "KOR", "ZWE")
```

```
#' At an interior vertex: the angle between incoming and outgoing steps, and the  
#' summed length of those two steps.
```

```
bend_stats <- function(P, i) {  
  a <- P[i, ] - P[i - 1, ]; b <- P[i + 1, ] - P[i, ]  
  na <- sqrt(sum(a^2)); nb <- sqrt(sum(b^2))  
  ang <- if (na == 0 || nb == 0) NA_real_ else acos(pmax(pmin(sum(a * b) / (na * nb), 1), -1)  
  c(turn = ang, mag = na + nb)
```

```

}

turns_of <- function(P) {
  k <- nrow(P); if (k < 3) return(numeric(0))
  sapply(2:(k - 1), function(i) bend_stats(P, i)["turn"])
}

build_turns <- function(df, id) {
  df %>% arrange(country, wave) %>% group_by(country) %>% group_split() %>%
    map_dfr(function(d) {
      P <- as.matrix(d[, c("Dim1", "Dim2")])
      tv <- turns_of(P)
      if (!length(tv)) return(NULL)
      tibble(source = id, country = d$country[1],
              vertex_wave = d$wave[2:(nrow(d) - 1)], turn_deg = tv)
    })
}

#' ## Turn profiles, both sides built by identical code

truth_df <- c2 %>% filter(country %in% REV_ISO) %>%
  distinct(country, wave, Dim1 = gt1, Dim2 = gt2) %>% arrange(country, wave)
truth_turns <- build_turns(truth_df, "truth")

model_turns <- c2 %>% filter(country %in% REV_ISO) %>%
  select(model, country, wave, Dim1, Dim2) %>%
  group_by(model) %>% group_split() %>%
  map_dfr(function(d) build_turns(d, d$model[1]) %>% mutate(model = d$model[1]))

saveRDS(list(truth = truth_turns, model = model_turns), "reversal_turns.rds")

#' The true reversal for each country is the vertex with the largest turn in the
#' ground truth. That wave is the target each model must hit.

truth_peak <- truth_df %>% group_by(country) %>% group_split() %>%
  map_dfr(function(d) {
    P <- as.matrix(d[, c("Dim1", "Dim2")]); k <- nrow(P)
    st <- t(sapply(2:(k - 1), function(i) bend_stats(P, i)))
    j <- which.max(st[, "turn"])
    tibble(country = d$country[1], true_bend_wave = d$wave[1 + j],
            true_turn = round(st[j, "turn"]), true_mag = st[j, "mag"],
            k_waves = k)
  })

```

```

})

cat("\n=== true reversals: the target each model must reproduce ===\n")

=== true reversals: the target each model must reproduce ===

print(truth_peak %>% arrange(desc(true_turn)))

# A tibble: 6 x 5
  country true_bend_wave true_turn true_mag k_waves
  <chr>         <dbl>     <dbl>    <dbl>   <int>
1 IND             6       169    0.366     4
2 JOR             6       169    0.255     4
3 IDN             5       165    0.293     3
4 ZWE             6       146    0.493     3
5 JPN             6       129    0.244     4
6 KOR             5       129    0.162     4

#' ## Each model's turn at the true bend wave

model_bend <- c2 %>% filter(country %in% REV_ISO) %>%
  select(model, country, wave, Dim1, Dim2) %>% arrange(model, country, wave) %>%
  group_by(model, country) %>% group_split() %>%
  map_dfr(function(d) {
    P <- as.matrix(d[, c("Dim1", "Dim2")]); k <- nrow(P); if (k < 3) return(NULL)
    st <- data.frame(vertex_wave = d$wave[2:(k - 1)],
                     t(apply(2:(k - 1), function(i) bend_stats(P, i))))
    tibble(model = d$model[1], country = d$country[1], st = list(st))
  }) %>%
  inner_join(truth_peak, by = "country") %>%
  mutate(row = map2(st, true_bend_wave, ~ .x[.x$vertex_wave == .y, ][1, ])) %>%
  transmute(model, country, true_turn, true_mag,
            model_max_turn = map_dbl(st, ~ round(max(.x$turn, na.rm = TRUE))),
            model_turn = round(map_dbl(row, "turn")),
            model_mag = map_dbl(row, "mag"),
            mag_ratio = round(model_mag / true_mag, 2))

#' ## Verdict
#'
```

```
#' A genuine reproduction requires both a sharp turn ( 120°) and a journey at
#' least half as long as the real one. The intermediate category - sharp but tiny
#' - is the one that matters interpretively: the shape of the reversal is there,
#' the movement is not.
```

```
comp <- model_bend %>%
  mutate(hit = case_when(
    model_turn >= 120 & mag_ratio >= 0.5 ~ "REAL reversal",
    model_turn >= 120 & mag_ratio < 0.5 ~ "sharp but TINY (noise)",
    model_turn < 120 & model_turn >= 60 ~ "partial bend",
    TRUE ~ "no bend")) %>%
  arrange(country, model)
saveRDS(comp, "reversal_hits.rds")

cat("\n=== reversal hits with magnitude guard ===\n")
```

```
=== reversal hits with magnitude guard ===
```

```
print(comp %>% mutate(true_mag = round(true_mag, 2), model_mag = round(model_mag, 2)), n = Inf)
```

```
# A tibble: 24 x 9
```

	model	country	true_turn	true_mag	model_max_turn	model_turn	model_mag
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Qwen/Qwen3.6--	IDN	165	0.29	69	69	0.15
2	claude-opus-4~	IDN	165	0.29	26	26	0.16
3	gemini-3.6-fl~	IDN	165	0.29	49	49	0.13
4	gpt-5.5	IDN	165	0.29	174	174	0.16
5	Qwen/Qwen3.6--	IND	169	0.37	43	43	0.25
6	claude-opus-4~	IND	169	0.37	162	162	0.18
7	gemini-3.6-fl~	IND	169	0.37	130	67	0.17
8	gpt-5.5	IND	169	0.37	82	15	0.12
9	Qwen/Qwen3.6--	JOR	169	0.25	99	99	0.06
10	claude-opus-4~	JOR	169	0.25	72	72	0.15
11	gemini-3.6-fl~	JOR	169	0.25	179	179	0.14
12	gpt-5.5	JOR	169	0.25	62	9	0.06
13	Qwen/Qwen3.6--	JPN	129	0.24	38	38	0.11
14	claude-opus-4~	JPN	129	0.24	173	173	0.19
15	gemini-3.6-fl~	JPN	129	0.24	170	169	0.11
16	gpt-5.5	JPN	129	0.24	138	138	0.15
17	Qwen/Qwen3.6--	KOR	129	0.16	22	7	0.14

18	claude-opus-4~	KOR	129	0.16	105	105	0.14
19	gemini-3.6-fl~	KOR	129	0.16	47	47	0.1
20	gpt-5.5	KOR	129	0.16	28	2	0.12
21	Qwen/Qwen3.6~~	ZWE	146	0.49	36	36	0.12
22	claude-opus-4~	ZWE	146	0.49	150	150	0.09
23	gemini-3.6-fl~	ZWE	146	0.49	33	33	0.1
24	gpt-5.5	ZWE	146	0.49	59	59	0.08

i 2 more variables: mag_ratio <dbl>, hit <chr>

```
cat("\n=== which sharp turns survive the magnitude guard? ===\n")
```

=== which sharp turns survive the magnitude guard? ===

```
comp %>% filter(model_turn >= 120) %>%
  select(country, model, model_turn, mag_ratio, hit) %>% print(n = Inf)
```

A tibble: 7 x 5

	country	model	model_turn	mag_ratio	hit
	<chr>	<chr>	<dbl>	<dbl>	<chr>
1	IDN	gpt-5.5	174	0.54	REAL reversal
2	IND	claude-opus-4-8	162	0.48	sharp but TINY (noise)
3	JOR	gemini-3.6-flash	179	0.54	REAL reversal
4	JPN	claude-opus-4-8	173	0.79	REAL reversal
5	JPN	gemini-3.6-flash	169	0.44	sharp but TINY (noise)
6	JPN	gpt-5.5	138	0.61	REAL reversal
7	ZWE	claude-opus-4-8	150	0.19	sharp but TINY (noise)

```
cat("\n=== reproduction count, out of", nrow(comp), "country-model cases ===\n")
```

=== reproduction count, out of 24 country-model cases ===

```
comp %>% count(hit) %>% print()
```

A tibble: 4 x 2

hit	n
<chr>	<int>
1 REAL reversal	4

2 no bend	12
3 partial bend	5
4 sharp but TINY (noise)	3

```
#' Distance, not direction, is what fails: across all cases the models move a
#' fraction of the true distance through the bend, the same halving of the rate
#' seen in script 07, appearing at exactly the moment when a country's movement
#' is what makes the reversal a reversal.
```

```
cat("\n=== median turn and median magnitude ratio at the true bend, by model ===\n")
```

```
=== median turn and median magnitude ratio at the true bend, by model ===
```

```
comp %>% group_by(model) %>%
  summarise(median_turn = median(model_turn, na.rm = TRUE),
            median_mag_ratio = round(median(mag_ratio, na.rm = TRUE), 2),
            max_turn = max(model_turn, na.rm = TRUE), .groups = "drop") %>% print()
```

```
# A tibble: 4 x 4
```

	model <chr>	median_turn <dbl>	median_mag_ratio <dbl>	max_turn <dbl>
1	Qwen/Qwen3.6-Plus	40.5	0.46	99
2	claude-opus-4-8	128.	0.56	173
3	gemini-3.6-flash	58	0.45	179
4	gpt-5.5	37	0.44	174

```
cat("true median turn:", median(truth_peak$true_turn), "\n")
```

```
true median turn: 155.5
```

```
cat("median mag_ratio across all cases:", round(median(comp$mag_ratio, na.rm = TRUE), 2), "\n")
```

```
median mag_ratio across all cases: 0.49
```